

Báo cáo tổng hợp HW04 - Automation Testing

Mục lục

- Thông tin bài tập
- Phạm vi và tài liệu thiết kế test case
- Chuyển đổi test case thành script tự động
- Thiết kế data-driven và cấu trúc automation
- Kết quả chạy thử automation
- Bug report và GitHub Issues
- Rà soát và phân tích khoảng trống script do AI tạo
- Trách nhiệm cuối cùng với bộ script
- Danh sách tài liệu nộp kèm
- Kết luận

1. Thông tin bài tập

- Môn học:** Software Testing
- Bài tập:** Homework 04 - Automation Testing
- Hệ thống kiểm thử:** EShop SUT
- Repository:** <https://github.com/yuran1811/hcmus-sw-testing--eshop-sut/tree/hw4/23127115-mqtan>
- Sinh viên:** Mạch Quốc Tấn
- MSSV:** 23127115
- Lớp:** 23KTPM2
- Ngày cập nhật báo cáo:** 10/08/2026

Báo cáo này tổng hợp toàn bộ quy trình tự động hóa kiểm thử cho HW04: từ thiết kế test case bằng Domain Testing và Boundary Value Analysis, chuyển đổi test case thành Playwright scripts, tổ chức dữ liệu data-driven, chạy đa trình duyệt, phân tích kết quả, lập bug report, và rà soát chất lượng script do AI hỗ trợ tạo ra.

2. Phạm vi và tài liệu thiết kế test case

2.1 Tính năng được chọn

Ba tính năng web được chọn để tự động hóa:

Tính năng	Requirement	Lý do chọn
Product List & Search	FR-05	Luồng cốt lõi để người dùng xem danh sách sản phẩm, tìm kiếm và tương tác với sản phẩm.
Checkout	FR-08	Luồng nghiệp vụ quan trọng liên quan đến giỏ hàng, thanh toán, tổng tiền và dữ liệu đơn hàng.
Category Management	FR-14	Luồng quản trị dữ liệu cần kiểm tra CRUD, validate input, phân quyền và tính toàn vẹn dữ liệu.

2.2 Tài liệu thiết kế test case

Thiết kế test case được thực hiện trước khi viết automation script. Hai tài liệu thiết kế chính:

- [Domain_Testing_Report.md](#)
- [Boundary_Value_Analysis_Report.md](#)

Hai báo cáo này mô tả phân vùng tương đương, miền dữ liệu hợp lệ/không hợp lệ, giá trị biên, điều kiện tiền đề, expected result và lý do chọn test data cho từng tính năng.

2.3 Tổng số test case thiết kế

Tính năng	Số test case	Kỹ thuật chính
Category Management	28	Domain Testing, Equivalence Partitioning, Boundary Value Analysis, Security/API validation
Checkout	22	Domain Testing, Equivalence Partitioning, Boundary Value Analysis, API/UI validation
Product List & Search	29	Domain Testing, Equivalence Partitioning, Boundary Value Analysis, UI/SEO checks
Tổng cộng	79	-

3. Chuyển đổi test case thành script tự động

Quá trình chuyển đổi test case thành Playwright script được thực hiện theo hướng có kiểm soát: test case thiết kế là nguồn chính, AI chỉ hỗ trợ tạo nháp script, còn tester rà soát lại mapping, dữ liệu, assertion và kết quả mong đợi.

3.1 Nguyên tắc chuyển đổi

- **Tách dữ liệu khỏi logic test:** dữ liệu đầu vào và expected result được đưa vào các file JSON để script không bị hardcoded và dễ mở rộng.
- **Giữ mapping rõ ràng:** mỗi test case trong tài liệu đều được ánh xạ về một file `.spec.ts` cụ thể.
- **Không ép mọi thứ thành data-driven:** chỉ dùng data-driven khi nhiều test có cùng luồng thao tác; các test UI, navigation, phân quyền hoặc tích hợp phức tạp được viết riêng để dễ đọc.
- **Dùng TypeScript:** script được viết bằng TypeScript để giảm lỗi kiểu dữ liệu, đặc biệt khi parse JSON test data và response API.

- **Rà soát assertion:** mỗi expected result phải có assertion tương ứng, không chỉ kiểm tra request không crash hoặc page load thành công.

3.2 Data-driven và non-data-driven

Các test case được chia thành hai nhóm khi chuyển sang script:

Nhóm	Đặc điểm	Ví dụ
Data-driven	Nhiều test case có cùng luồng thao tác nhưng khác input/expected result. Dữ liệu được đặt trong JSON và duyệt bằng vòng lặp.	BVA độ dài tên danh mục, sai kiểu dữ liệu của <code>name</code> , biên độ dài từ khóa search, biên địa chỉ giao hàng.
Non-data-driven	Test case có luồng riêng, cần setup riêng hoặc assertion riêng; nếu ép vào JSON sẽ làm script khó đọc hơn.	Kiểm tra UI checkout, điều hướng logo, phân tách giỏ hàng user A/user B, kiểm tra security payload.

3.3 Mapping theo từng tính năng

FR-14 Category Management

Nhóm test case	Script tự động	Ghi chú chuyển đổi
TC-CATEGORY-001 đến TC-CATEGORY-006	<code>category-crud.spec.ts</code>	CRUD danh mục, thêm/xóa danh mục, validate tên rỗng/khoảng trắng và xóa ID không tồn tại.
TC-CATEGORY-007, 008, 010, 011, 018	<code>category-auth.spec.ts</code>	Kiểm tra thiếu token, token user thường, token sai định dạng hoặc hết hạn.
TC-CATEGORY-009, 012, 013, 014, 015, 019, 020	<code>category-crud.spec.ts</code>	Kiểm tra orphan record, thiếu <code>name</code> , <code>name</code> sai kiểu, Unicode, trùng tên, ID sai kiểu và idempotency.
TC-CATEGORY-016, 017	<code>category-security.spec.ts</code>	Kiểm tra HTML/XSS payload và SQL payload trong tên danh mục.
TC-CATEGORY-BVA-001 đến 008	<code>category-bva.spec.ts</code>	Kiểm tra biên độ dài tên danh mục và biên ID khi xóa.

FR-08 Checkout

Nhóm test case	Script tự động	Ghi chú chuyển đổi
TC-CHECKOUT-001 đến 010	<code>checkout-api.spec.ts</code>	Kiểm tra checkout API, token, giỏ hàng, tổng tiền, địa chỉ giao hàng và schema payload.
TC-CHECKOUT-011, 012	<code>checkout-ui.spec.ts</code>	Kiểm tra UI checkout hiển thị thông tin sản phẩm và tổng tiền không được sửa từ client.
TC-CHECKOUT-013, 014, 015	<code>checkout-api.spec.ts</code>	Kiểm tra server tự tính total_amount khi client không gửi, items/giá giả mạo và phân tách dữ liệu giữa người dùng.
TC-CHECKOUT-BVA-001 đến 007	<code>checkout-bva.spec.ts</code>	Kiểm tra biên số lượng giỏ hàng và độ dài/kiểu dữ liệu địa chỉ giao hàng.

FR-05 Product List & Search

Nhóm test case	Script tự động	Ghi chú chuyển đổi
TC-PLAS-001 đến 006, 008 đến 014	<code>plas-ep.spec.ts</code>	Kiểm tra danh sách sản phẩm, từ khóa hợp lệ/không hợp lệ, tiếng Việt, XSS, SQL payload, trim và empty state.
TC-PLAS-007, 015 đến 019	<code>plas-ui.spec.ts</code>	Kiểm tra hiển thị card sản phẩm, ảnh, giá, nút chi tiết, thêm giỏ hàng, logo, footer count và loading state.
TC-PLAS-BVA-001 đến 010	<code>plas-bva.spec.ts</code>	Kiểm tra biên độ dài từ khóa, overflow, SQL/SQLite error và các mốc ngay dưới/ngay trên biên.

3.4 Vì sao số test chạy nhiều hơn số test case

Một test case tài liệu không nhất thiết tương ứng đúng một test execution trong Playwright. Có test case được triển khai bằng vòng lặp data-driven với nhiều dòng dữ liệu, có test case được chạy trên 3 browser, và có test case cần nhiều assertion để kiểm tra cả API lẫn UI. Vì vậy số lượt chạy trong HTML report có thể lớn hơn số test case thiết kế. Đây là hành vi mong muốn của automation, không phải lỗi đếm sai.

4. Thiết kế data-driven và cấu trúc automation

4.1 Cấu trúc thư mục

Automation scripts nằm trong:

- [Category scripts](#)
- [Checkout scripts](#)
- [Product List & Search scripts](#)

Mỗi suite có cấu trúc riêng gồm `tests/`, `data/`, cấu hình Playwright, HTML report và test-results.

4.2 Data-driven testing

Dữ liệu kiểm thử được tách khỏi logic test để tránh hardcode và giảm lặp code:

Tính năng	File dữ liệu chính	Cách dùng
Category	<code>category/data/category-test-data.json</code>	Dữ liệu CRUD, auth, BVA, kiểu dữ liệu sai, security payload.
Checkout	<code>checkout/data/checkout-test-data.json</code>	Dữ liệu checkout API, total_amount, shipping_address, BVA giỏ hàng/địa chỉ.
Product List & Search	<code>product-list-and-search/data/plas-test-data.json</code>	Dữ liệu tìm kiếm hợp lệ/không hợp lệ, SQL/XSS payload, biên độ dài từ khóa.

Các script dùng vòng lặp trên data file khi nhiều test case có cùng luồng xử lý nhưng khác input/expected result. Những test có luồng riêng như kiểm tra UI, điều hướng, hoặc phân tách dữ liệu giữa user được viết riêng để dễ đọc và dễ debug hơn.

4.3 Chạy đa trình duyệt và báo cáo HTML

Mỗi suite được cấu hình chạy trên 3 browser:

- Chromium
- Firefox
- WebKit

HTML report đã được cập nhật để hiển thị rõ thông tin người chạy:

- **Run by:** `23127115`
- **Generated at:** timestamp ISO tại thời điểm tạo report

Report nằm tại:

- [Category HTML report](#)
- [Checkout HTML report](#)
- [Product List & Search HTML report](#)

5. Kết quả chạy thử automation

5.1 Tổng quan theo test case

Số liệu mới nhất được đồng bộ trong:

- [Automation test summary](#)
- [Automation traceability matrix](#)

Chỉ số	Số lượng	Tỷ lệ
Tổng số test case đã thiết kế	79	100,0%
Test case đã cập nhật trạng thái	79	100,0%
Pass	41	51,9%
Fail	38	48,1%

5.2 Tổng quan theo browser runs

Bộ test	Browser runs	Passed runs	Failed runs
Category	102	58	44
Checkout	87	45	42
Product List & Search	87	45	42
Tổng cộng	276	148	128

5.3 Cách hiểu số lượng test chạy và số lượng bug

Số test chạy trong Playwright **không bằng** số test case tài liệu vì:

- Mỗi test được chạy trên 3 browser nên một logical test có thể sinh 3 browser runs.
- Một số test case được tách thành nhiều biến thể data-driven, ví dụ sai kiểu dữ liệu hoặc nhiều mốc biên.
- Một test case có thể có nhiều assertion trong cùng script để kiểm tra cả response, dữ liệu lưu trữ và UI.
- Một bug có thể làm nhiều test fail trên nhiều browser.
- Nhiều fail có thể cùng quy về một bug gốc.

Vì vậy **128** failed browser runs **không có nghĩa là có 128 bug**. Sau khi phân tích và gom nhóm theo nguyên nhân gốc, bộ automation ghi nhận **28** bug report hợp lệ.

6. Bug report và GitHub Issues

Bug report tổng hợp được cập nhật tại:

- [Bug_Report.md](#)
- [tests/bug-reports/automation](#)

Tổng số bug report automation:

Module	Bug report	GitHub Issue	Bảng chứng
Category	10	#237 - #246	API assertion log
Checkout	10	#247 - #256	API log + UI screenshot
Product List & Search	8	#257 - #264	Playwright screenshot
Tổng cộng	28	#237 - #264	-

Các bug report đã được cập nhật link GitHub Issue trực tiếp. Những lỗi API không có screenshot trình duyệt được giữ bằng chứng bằng HTTP request/response log và Playwright assertion log; các lỗi UI có screenshot markdown trực tiếp hoặc raw image URL để hiển thị được trên GitHub Issue.

7. Rà soát và phân tích khoảng trống script do AI tạo

Phần này tổng hợp trực tiếp quá trình rà soát script do AI hỗ trợ tạo ra. Mục tiêu của bước này là không chấp nhận script ở trạng thái "AI sinh ra là xong", mà phải kiểm tra lại từng assertion, từng wait, từng selector và từng expected result để phân biệt lỗi thật của SUT với lỗi do script yếu.

7.1 Những vấn đề AI làm chưa tốt

Nhóm vấn đề	Ví dụ đã gặp	Vì sao AI bỏ sót
Assertion yếu hoặc thiếu	Ban đầu FR-05 chỉ kiểm tra trang không crash, chưa kiểm tra đủ <code>h1</code> , <code>alt</code> , định dạng giá và empty state.	Prompt ban đầu tập trung vào flow chính, chưa yêu cầu AI chứng minh từng expected result bằng assertion cụ thể.
Wait chưa ổn định	Search từng chờ <code>domcontentloaded</code> thay vì chờ response <code>/api/products?search= ...</code> ; loading state có thể fail do race condition.	AI thường sinh wait tổng quát vì không biết timing thật của SUT và API nào là tín hiệu hoàn tất đáng tin cậy.
Selector có nguy cơ fragile	Một số locator UI ban đầu dựa vào text hoặc cấu trúc DOM để thay đổi thay vì locator theo vai trò/ý nghĩa.	AI chỉ nhìn mô tả tính năng, không có đủ ngữ cảnh DOM thật hoặc design system ổn định.
Thiếu edge case	Category ban đầu chưa bắt chặt delete ID sai kiểu, ID ngoài biên, thiếu <code>name</code> , <code>name</code> sai kiểu; Checkout thiếu kiểm tra total_amount/items giả mạo.	Model có xu hướng sinh happy path và vài negative path phổ biến, nhưng bỏ sót lỗi bảo mật hoặc integrity nếu prompt không ép theo threat model.

7.2 Các vấn đề cụ thể đã phát hiện

7.2.1 Automation pass giả ở FR-05 Product List & Search

Ban đầu một số test case FR-05 vẫn pass dù khi kiểm thử thủ công có thể thấy lỗi. Nguyên nhân chính là assertion còn quá mềm: script chỉ kiểm tra trang không crash hoặc có danh sách sản phẩm, nhưng chưa kiểm tra đầy đủ các dấu hiệu lỗi như số lượng thẻ `h1`, thuộc tính `alt` của ảnh, định dạng giá và thông điệp empty state.

Ngoài ra, hàm search ban đầu chờ `domContentLoaded`, trong khi điều kiện hoàn tất thật của tính năng là response API `/api/products?search= ...`. Vì vậy script có thể đọc DOM ở thời điểm chưa ổn định hoặc bỏ qua lỗi timing của response.

7.2.2 Loading state và reload bị nhiễu timing

`TC-PLAS-019` từng có khả năng fail do timing sau `reload()`. Đây không phải bug thật của SUT mà là lỗi cách chờ trong automation. Nếu giữ nguyên, báo cáo sẽ ghi nhận sai lỗi ứng dụng trong khi thực tế script chưa đủ ổn định.

7.2.3 Category delete và validate input quá mềm

Ở Category, AI tạo script ban đầu có xu hướng chấp nhận nhiều response như "hợp lệ" để tránh fail. Ví dụ `TC-CATEGORY-006` từng chấp nhận delete danh mục không tồn tại như một kết quả thành công. `TC-CATEGORY-009` chỉ kiểm tra status code sau khi xóa danh mục, nên chưa phát hiện orphan record ở sản phẩm liên kết.

Các case validate input như thiếu `name`, `name` sai kiểu dữ liệu, ID sai kiểu hoặc ID ngoài biên cũng cần assertion chặt hơn để bắt lỗi backend thay vì chỉ kiểm tra request không crash.

7.2.4 Checkout thiếu kiểm tra bảo mật dữ liệu

Ở Checkout, script AI ban đầu tập trung nhiều vào luồng hợp lệ, nhưng chưa đủ mạnh ở các tình huống bảo mật và tính toàn vẹn dữ liệu. Các case cần được siết lại gồm token JWT không hợp lệ, giỏ hàng trống, `total_amount` giả mạo thấp/cao hơn giá thực tế, items/giá giả mạo từ client, `shipping_address` sai kiểu dữ liệu, và kiểm tra checkout của user A không làm ảnh hưởng giỏ hàng user B.

7.3 Các chỉnh sửa sau rà soát

- Siết assertion FR-05 để kiểm tra đúng 1 thẻ `h1`, ảnh có `alt`, giá có ký hiệu `đ`, empty state và lỗi SQL/SQLite.
- Sửa `ProductListPage.search()` để chờ đúng response API thay vì wait mơ hồ.
- Tách lỗi timing khỏi lỗi thật, đặc biệt với loading/reload state.
- Siết Category API để không chấp nhận delete nonexistent như success và kiểm tra orphan records sau khi xóa danh mục.
- Siết Checkout API để kiểm tra token không hợp lệ, giỏ hàng trống, total_amount/items giả mạo, shipping_address sai kiểu và phân tách dữ liệu giữa user.

7.4 Kết quả sau khi sửa và chạy lại

Sau khi rà soát và sửa script, toàn bộ 3 suite được chạy lại trên 3 trình duyệt. Product List & Search và Checkout được chạy lại lần gần nhất ngày 10/08/2026. Kết quả cuối cùng:

Suite	Passed runs	Failed runs	Tổng browser runs
Product List & Search	45	42	87
Category	58	44	102
Checkout	45	42	87
Tổng cộng	148	128	276

Các fail còn lại được phân tích lại theo nguyên nhân gốc và gom thành 28 bug report automation. Những lỗi không phải bug thật, ví dụ wait/reload timing, đã được sửa ở script thay vì đưa vào bug report.

7.5 Kết luận gap analysis

AI hữu ích trong việc khởi tạo skeleton test, chuyển test case thành code ban đầu và đề xuất cấu trúc data-driven. Tuy nhiên AI không đủ tin cậy nếu để tự quyết định assertion cuối cùng. Các vấn đề quan trọng như phân tích nguyên nhân gốc, phân biệt script bug với SUT bug, chọn wait ổn định và bảo đảm mỗi expected result có assertion tương ứng đều cần tester rà soát thủ công.

8. Trách nhiệm cuối cùng với bộ script

Bộ script cuối cùng không được xem là sản phẩm do AI tự quyết định. AI chỉ được dùng như công cụ hỗ trợ tạo nháp, gợi ý cấu trúc, gợi ý refactor và hỗ trợ rà soát.

Các quyết định cuối cùng do tester chịu trách nhiệm:

- Chọn test case nào được tự động hóa.
- Xác định expected result và oracle kiểm thử.
- Sửa selector, wait và assertion cho ổn định.
- Phân loại fail nào là bug thật của SUT, fail nào là lỗi script.
- Gom các fail trùng nguyên nhân thành bug report hợp lý.
- Cập nhật traceability, summary và GitHub Issues theo kết quả chạy thật.

9. Danh sách tài liệu nộp kèm

Hạng mục	Tài liệu
Thiết kế Domain Testing	Domain_Testing_Report.md
Thiết kế Boundary Value Analysis	Boundary_Value_Analysis_Report.md
Bug report tổng hợp	Bug_Report.md
Automation test summary	test-summary-report.md
Automation traceability matrix	traceability-matrix.md
Automation scripts và artifacts	tests/test-runs/automation/scripts
Bug reports chi tiết	tests/bug-reports/automation
AI audit log	AI_Audit_Report.md
AI critique	AI_Critique.md

10. Kết luận

Bộ kiểm thử tự động đã bao phủ 3 tính năng chính FR-05, FR-08 và FR-14 với 79 test case thiết kế, chạy đa trình duyệt trên Chromium, Firefox và WebKit. Kết quả theo test case/truy vết mới nhất ghi nhận 41 pass và 38 fail; kết quả browser runs ghi

nhận 276 lượt chạy, trong đó 148 pass và 128 fail. Sau khi phân tích nguyên nhân gốc, các fail được gom thành 28 bug report automation và đã tạo GitHub Issues #237 đến #264.

Phần quan trọng nhất của quá trình này không chỉ là tạo được script Playwright, mà là rà soát và sửa script do AI hỗ trợ sinh ra. Các script cuối cùng đã được chỉnh để có assertion mạnh hơn, wait ổn định hơn, data-driven rõ ràng hơn và traceability nhất quán với test case, test run, bug report và GitHub Issue.